

Future Refactor Plan

Goal

Reduce operational risk without stopping product delivery.

Phase 1: Stabilize interfaces

- define canonical role resolution
- define canonical notification payload schema
- define canonical feed event schema
- document livestream lifecycle states and transitions

Phase 2: Split runtime concerns

- extract Socket.IO into a dedicated realtime gateway or at least a dedicated module boundary
- move cron jobs to a worker process
- move static store/blog delivery away from the social API process

Phase 3: Distributed state

- add Redis for Socket.IO adapter
- store presence and livestream room state in Redis
- add queue-backed notification fan-out

Phase 4: Data and feed hardening

- centralize counter reconciliation
- precompute or cache feed candidates
- reduce populate-heavy hot paths

Phase 5: Operational maturity

- add structured metrics for:
 - socket room counts
 - notification latency
 - feed query latency
 - cron job duration
 - reward issuance volume
- add health endpoints for external dependencies such as Mongo, OneSignal, Cloudinary, and Agora

Outcome

The target architecture should preserve current product behavior while making the platform safer to scale, easier to debug, and easier for both engineers and AI systems to reason about.